

Постановка задачи.

Ввести 10-15 целых чисел и построить из них бинарное дерево поиска. Выполнить симметричную прошивку бинарного дерева поиска. Обойти его согласно симметричному порядку следования элементов. Реализовать поиск и вставку элементов симметрично прошитого бинарного дерева.

Листинг программы:

```
#include <iostream>
#include <iomanip>
#include <string>
#include <stdio.h>
#include <stdlib.h>
using namespace std;

struct treeNode // Структура дерева
{
    int field; // поле данных
    bool ltag, rtag; // теги прошивочных нитей
    struct treeNode* left; // указатель на левое поддерево
    struct treeNode* right; // указатель на правое поддерево
};

treeNode* y = new treeNode; // указатель на предыдущий узел

// Вывод дерева (в инфиксной форме)
void displaySortedTree(treeNode* tree)
{
    if (tree != NULL)
    {
        displaySortedTree(tree->left);
        cout << tree->field;
        displaySortedTree(tree->right);
    }
}

// Добавление узла
struct treeNode* addNode(int x, treeNode* tree)
{
    if (tree == NULL)
    {
        tree = new treeNode;
        tree->field = x;
        tree->left = NULL;
        tree->right = NULL;
    }
    else
    {
        if (x < tree->field)
            tree->left = addNode(x, tree->left);
        if (x > tree->field)
            tree->right = addNode(x, tree->right);
    }
    return(tree);
}

// Левая прошивка дерева
void leftsew(treeNode* p) {
    if (p != nullptr) {
        if (p->left == nullptr) {
            p->ltag = false;
        }
    }
}
```

```

        p->left = y;
    }
    else
        p->ltag = true;
}
p = y;
}

// Правая прошивка дерева
void rightsew(treeNode* p) {
    if (y != nullptr) {
        if (y->right == nullptr) {
            y->rtag = false;
            y->right = p;
        }
        else
            y->rtag = true;
    }
    y = p;
}

// Добавление узла с поддержанием нитей
struct treeNode* addSimNode(int x, treeNode* tree)
{
    if (tree == NULL)
    {
        tree = new treeNode;
        tree->field = x;
        tree->left = NULL;
        tree->right = NULL;

        leftsew(tree);
        rightsew(tree);
    }
    else
    {
        if (x < tree->field)
            tree->left = addSimNode(x, tree->left);
        if (x > tree->field)
            tree->right = addSimNode(x, tree->right);
    }
    return(tree);
}

// Симметричный вывод дерева
void simPrint(treeNode* tree) {
    if (tree != nullptr) {
        simPrint(tree->left);
        leftsew(tree);
        rightsew(tree);
        cout << tree->field << " ";
        simPrint(tree->right);
        y = tree;
    }
}

// Поиск узла в дереве
treeNode* findNode(int x, treeNode* tree)
{
    if (tree == NULL)
        return NULL;
    if (tree->field == x)
        return tree;
}

```

```

    if (x <= tree->field)
    {
        if (tree->left != NULL)
            return findNode(x, tree->left);
        else
            return NULL;
    }
    else
    {
        if (tree->right)
            return findNode(x, tree->right);
        else
            return NULL;
    }
}

// Генерация случайного целого числа в диапазоне
int randInt(int min, int max) { return rand() % (max - min + 1) + min; }

// Валидация ввода целого числа
void validateInt(int& a, string varName)
{
    string sA = "";
    do
    {
        try
        {
            if (varName != "")
                cout << "Введите " << varName << ": ";
            cin >> sA;
            a = stoi(sA);
            break;
        }
        catch (invalid_argument ex) {
            cout << "Введенные данные не являются числом" << endl;
        }
        catch (out_of_range ex) {
            cout << "Введено слишком большое число" << endl;
        }
    } while (true);
}

int main()
{
    system("chcp 1251");
    system("cls");

    // Инициализация корня
    struct treeNode* root = 0;
    treeNode* HEAD = new treeNode;
    HEAD->left = root;
    HEAD->right = HEAD;

    cout << "Выберите пункт меню\n1. Ввести узлы вручную\n2. Заполнить случайными числами в диапазоне\n3. Заполнить заготовленными числами\n";
    int menu;
    validateInt(menu, "");
    switch (menu)
    {
    case 1:
    {
        int a, n;
        cout << "Сколько чисел будет вводиться? [10, 15]\n";
        do

```

```

    {
        validateInt(n, "n");
    } while (n < 10 || n > 15);

    for (int i = 0; i < n; i++)
    {
        cout << "Введите узел " << i + 1 << ": ";
        validateInt(a, "");
        root = addNode(a, root);
    }
}

break;
case 2:
    int min, max;
    cout << "Введите минимальное генерируемое число: \n";
    validateInt(min, "");
    cout << "Введите максимальное генерируемое число: \n";
    validateInt(max, "");
    for (int i = 0; i < 15; i++)
        root = addNode(randInt(min, max), root);
    break;
default:
    root = addNode(4, root);
    root = addNode(3, root);
    root = addNode(5, root);
    root = addNode(1, root);
    root = addNode(7, root);
    root = addNode(8, root);
    root = addNode(6, root);
    root = addNode(2, root);
    break;
}

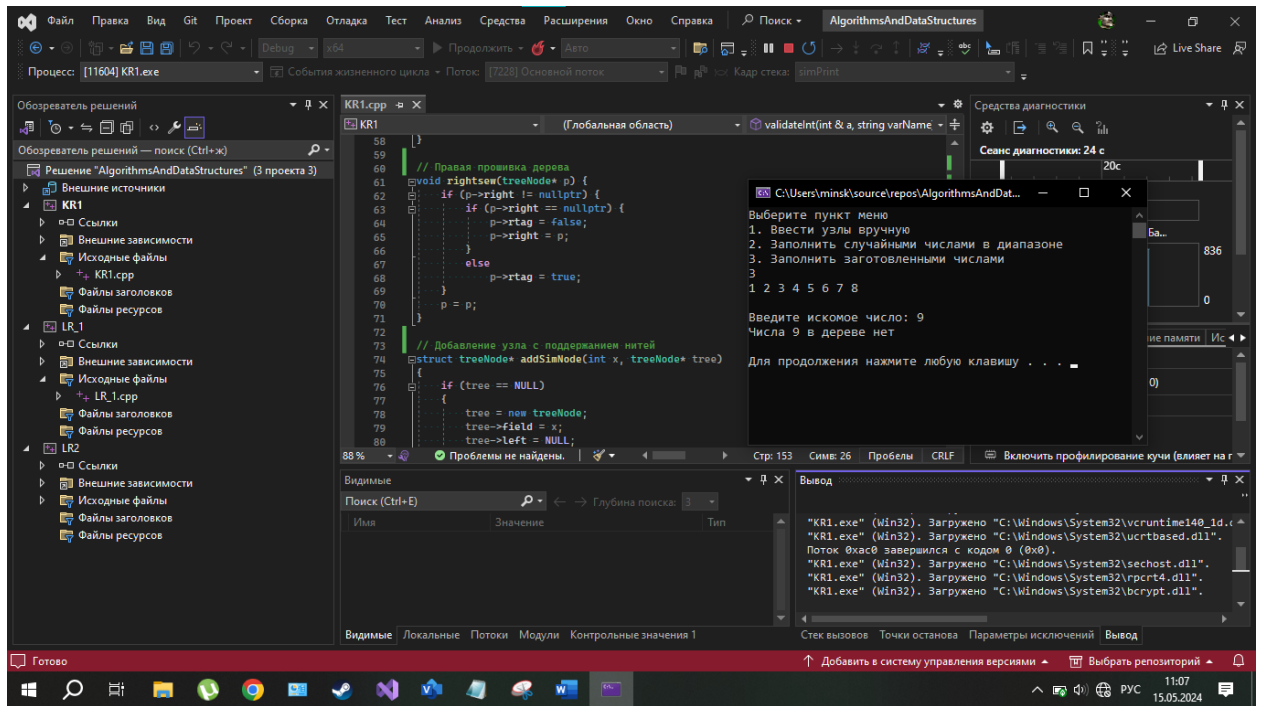
simPrint(root);
cout << endl << endl;

int num;
validateInt(num, "искомое число");
struct treeNode* tn1 = findNode(num, root);
if (tn1)
    cout << tn1->field << endl;
else
    cout << "Числа " << num << " в дереве нет" << endl;

cout << "\n";
system("pause");
return 0;
}

```

Результат выполнения программы (снимок экрана):



Ответы на контрольные вопросы:

1. Перечислите достоинства и недостатки прошивки бинарных деревьев.
Преимущества: быстрый обход, отсутствие необходимости в стеке, можно определить предшественника и преемника вершины.
Недостатки: Включение новой вершины в дерево занимает больше времени, т.к. необходимо поддерживать два типа связей: структурные и по нитям.
2. Чем AVL-дерево отличается от идеально сбалансированного бинарного дерева?
Отличие AVL-дерева от обычного дерева поиска заключается в том, что при включении и удалении элементов необходимо поддерживать сбалансированность дерева в целом. Для этого в каждый узел дерева добавляется одно вспомогательное поле, содержащее информацию о равновесности поддеревьев (показатель сбалансированности узла).